

Alternative au choix de `weights` avec `kmeans` ?

July 1, 2026

1 Correction d'une erreur

Sur la définition d'un nouvel attribut, je met un `max` au lieu d'un `min`, puisque l'idée est de minimiser ces indicateurs. Le `max` me garantit qu'à la fois sur les pixels dans l'ensemble et sur les autres, on se donne l'objectif d'une valeur faible.

On peut adapter les indicateurs à ce problème en changeant leur définition.

$$\text{taille du contour de A} \Rightarrow \max \frac{\text{taille du contour dans A}}{\sqrt{\text{taille de A}}}, \frac{\text{taille du contour extérieur à A}}{\sqrt{\text{taille extérieur à A}}} \quad (1)$$

$$\text{Nombre de points isolés de A} \Rightarrow \max \frac{\text{nombre de points isolés dans A}}{\sqrt{\text{taille de A}}}, \frac{\text{nombre de points isolés extérieur à A}}{\sqrt{\text{taille extérieur à A}}} \quad (2)$$

2 Adaptation d'un critère à un nombre restreint de pixels

Je considère les notations suivantes.

- $\mathcal{M}$ est l'ensemble des pixels de l'image.
- $\mathcal{M}'$ est l'ensemble restreint de pixels : $\mathcal{M}' \subset \mathcal{M}$.
- $N(m)$ est l'ensemble des voisins de m . J'appelle l'ensemble des voisins restreint à $\mathcal{M}'$: $N(m) \cap \mathcal{M}'$.
- $\mathcal{G}$ est l'ensemble des classes, leur nombre est $G = |\mathcal{G}|$.
- F est un résultat de segmentation au sens où c'est une application de $\mathcal{M}$ dans $\mathcal{G}$.
- $\mathbf{1}(\dots)$ vaut 1 si les points de suspension sont remplacés par une vraie proposition et 0 sinon.

La définition **CE** est ainsi adaptée

$$\begin{aligned} \text{CE}(F) &= \frac{1}{|\mathcal{M}|} \sum_{m \in \mathcal{M}} \mathbf{1}(\exists m' \in N(m), F(m') \neq F(m)) \\ \text{devient } \text{CE}_{\mathcal{M}'}(F) &= \frac{1}{|\mathcal{M}'|} \sum_{m \in \mathcal{M}'} \mathbf{1}(\exists m' \in N(m) \cap \mathcal{M}', F(m') \neq F(m)) \end{aligned} \quad (3)$$

La définition de **CIP** est ainsi adaptée

$$\begin{aligned} \text{CIP}(F) &= \frac{1}{|\mathcal{M}|} \sum_{m \in \mathcal{M}} \mathbf{1}(\forall m' \in N(m), F(m') = F(m)) \\ \text{devient } \text{CIP}_{\mathcal{M}'}(F) &= \frac{1}{|\mathcal{M}'|} \sum_{m \in \mathcal{M}'} \mathbf{1}(\forall m' \in N(m) \cap \mathcal{M}', F(m') = F(m)) \end{aligned} \quad (4)$$

3 Utilisation d'un seuillage pour construire un arbre de décision

L'utilisation d'un arbre de décision est une idée classique :

https://fr.wikipedia.org/wiki/Arbre_de_d%C3%A9cision.

Et cette technique est utilisée en apprentissage :

[https://fr.wikipedia.org/wiki/Arbre_de_d%C3%A9cision_\(apprentissage\)](https://fr.wikipedia.org/wiki/Arbre_de_d%C3%A9cision_(apprentissage))

Je ne peux pas utiliser exactement la même technique, puisque je ne dispose pas d'ensemble d'apprentissage. Mais inspiré par [discussion_N230901.pdf](#), je propose de construire un arbre en partant de la racine et pour chaque noeud, de parcourir les différentes longueurs d'onde, de chercher des seuils et de choisir celui qui minimise un critère (**ce**, **cip** ou autre). À chaque itération, le noeud considéré est celui qui concerne le plus de pixels. Lorsque le critère est appliqué à un noeud qui n'est pas le noeud racine, le critère ne doit pas s'appliquer à l'ensemble de l'image puisque pour ce noeud, seul une partie des pixels sont concernés. Il suffit pour cela d'adapter les notions de voisinages en se restreignant aux pixels voisins est qui sont concernés. Ainsi si un noeud concerne un ensemble de pixels incluant un point isolé, ce pixel n'est plus considéré comme isolé pour l'application du critère.

J'introduis des nouvelles notations.

- $\mathcal{N}$ l'ensemble des noeuds d'un arbre.
- n_0 le noeud racine de l'arbre.
- $\mathcal{N}_t$ l'ensemble des noeuds terminaux d'un arbre. On a toujours $\mathcal{N}_t \subset \mathcal{N}$. Au début $\mathcal{N}_t = \{n_0\}$.
- $\mathcal{M}_n$ l'ensemble des pixels du noeud n : $\mathcal{M}_{n_0} = \mathcal{M}$.
- n_F est le noeud fils de n pour lequel la condition n'est pas vérifiée et n_V est le noeud fils de n pour lequel la condition est vérifiée.
- $I_k(m) \leq s_n$ est le critère avec s_n un seuil. $I_k(m)$ est l'intensité associée à la longueur d'onde k pour le pixel m .

$$\begin{aligned}\mathcal{M}_{n_F} &= \{m \in \mathcal{M}_n \mid I_k(m) > s_n\} \\ \mathcal{M}_{n_V} &= \{m \in \mathcal{M}_n \mid I_k(m) \leq s_n\}\end{aligned}\tag{5}$$

Et on a $\mathcal{M}_{n_F} \cap \mathcal{M}_{n_V} = \emptyset$ et $\mathcal{M}_{n_F} \cup \mathcal{M}_{n_V} = \mathcal{M}_n$

- $A(\mathcal{M}_n)$ est la valeur du score associée au sous-ensemble de pixels $\mathcal{M}_n$, A peut désigner **CE**, **CIP** ou tout autre score à minimiser.

L'algorithme considéré est le suivant.

1. Je choisis au départ le noeud terminal contenant le plus de pixels.

$$n = \arg \max_{n \in \mathcal{N}_t} |\mathcal{M}_n|\tag{6}$$

2. Je parcours l'ensemble des longueur d'onde $k \in \{0, \dots, K-1\}$.

- (a) Je note $v_0 \dots v_{L-1}$ les valeurs distinctes des $\mathcal{M}_n$ pour la longueur d'onde k .
- (b) Je cherche $\hat{l}_k$ qui minimise le score associé au seuillage par v_l pour la longueur d'onde k .

$$\hat{l}_k = \arg \min_{l \in \{0 \dots L-1\}} A(\{m \in \mathcal{M}_n, I_k(m) \leq v_l\})\tag{7}$$

- (c) Je note $a_k = A(\{m \in \mathcal{M}_n, I_k(m) \leq v_{\hat{l}_k}\})$

3. Je choisis k minimisant a_k .
4. Je crée deux nouveaux noeuds, n_F et n_V contenant les pixels de $\mathcal{M}_n$ vérifiant respectivement la condition $I_k(m) > v_{\hat{l}_k}$ et $I_k(m) \leq v_{\hat{l}_k}$
5. Si le nombre de noeuds terminaux est inférieur strictement à G alors je retourne à la première étape.

L'arbre obtenu n'est pas forcément une pyramide, il peut avoir la structure d'une échelle.

4 Génération d'une forêt à partir d'un arbre

On génère T arbres, chacun à partir de la méthode décrit plus haut. Pour que les arbres soient différents les uns des autres on applique une pondération aléatoire sur les intensités de chaque longueur d'onde. Les G terminaisons des T arbres définissent T segmentation en G régions, que l'on souhaiterait fusionner en une segmentation en G régions. Cette objectif s'appelle *consensus clustering* et c'est un sujet très étudié.

La première étape consiste à définir les H briques de bases ω_h de la segmentation finale avec $H \geq G$. On parcourt les pixels de l'image et pour chaque nouveau pixel, on considère l'intersection de tous les cluster qui contiennent ce pixels.

1. On initialise $H := 1$ et $\omega = \{\omega_h | h < H\} = \emptyset$
2. On parcourt les pixels de l'image $m \in \mathcal{M}$.
 - (a) Si $\{m' \in \mathcal{M} | \forall t, F_t(m) = F_t(m')\}$ n'est pas inclu dans ω on le rajoute en l'appelant ω_H et on incrémente H .

Par construction on a $F_t(\omega_h)$ est un ensemble contenant une seule valeur.

La seconde étape est de calculer les distances entre les différentes briques de base.

$$D_{hh'} = \frac{1}{T} \sum_t \mathbf{1}(F_t(\omega_h) = F_t(\omega_{h'})) \quad (8)$$

La troisième étape consiste à regrouper les H briques de base en G cluster de façon à minimiser les distances entre briques de base appartenant à des cluster différents. J'utilise l'algorithme suivant appelé aussi *Iterative Pairwise Consensus* qui ressemble au **kmeans**. Il s'agit d'initialiser avec une partition aléatoire en G cluster. Puis chaque brique de base est affectée au cluster pour lequel, il est le plus proche en moyenne des briques de bases incluses dans ce cluster. Ceci définit une nouvelle partition. On recommence cette opération tant que la partition n'est plus modifiée.

1. Choix aléatoire d'une façon de regrouper les H briques dans G cluster. C'est le choix d'une fonction F_0 de $\{0 \dots H - 1\}$ dans $\{0 \dots G - 1\}$.
2. Je repète la tâche suivante jusqu'à ce que $F_{t'+1} = F_t$:
 - (a) Je définis $F'(h) = \frac{1}{|\{h' | F_t(h') = F_t(h)\}|} \sum_{F_t(h') = F_t(h)} D_{hh'}$
 - (b) La partition de F_{t+1} est définie par

$$F_{t+1}(h) = \arg \min_{h'} F'(h) \quad (9)$$